

Unidad 5

POO Avanzada

Contenido

2

Contenido:

- ▶ Métodos: equals, hashCode, toString
- ▶ Uso, Agregación y Composición
- ▶ Herencia
- ▶ Polimorfismo
- ▶ Clases Abstractas

Métodos útiles

3

- ▶ Existen métodos comunes a todas las clases que son muy útiles y siempre podemos utilizar:
 - ▶ **equals** y **hashCode** : se utilizan conjuntamente, están relacionados
 - ▶ **toString**: permite representar un objeto (contenido)

equals - hashCode

4

► equals

- Compara dos objetos
- Devuelve *true* si los objetos son iguales
- Se puede personalizar, concretar cómo realizar la comparación.

► hashCode

- Calcula y devuelve un valor entero que 'resume' el objeto
- En una misma ejecución el *hashCode* de un objeto devuelve siempre el mismo entero
- Si dos objetos son iguales (confirmado con *equals*), ambos devuelven el mismo *hashCode*.

► toString

- Devuelve un String que representa el contenido del objeto
- No tiene parámetros.
- Una vez implementado para una clase en concreto, se podrán imprimir los objetos en una cadena de texto:

```
System.out.println(v1);
```

```
// imprime Vehiculo [marca=SEAT, modelo=Ibiza, anyo=2009]
```

Asociación entre clases

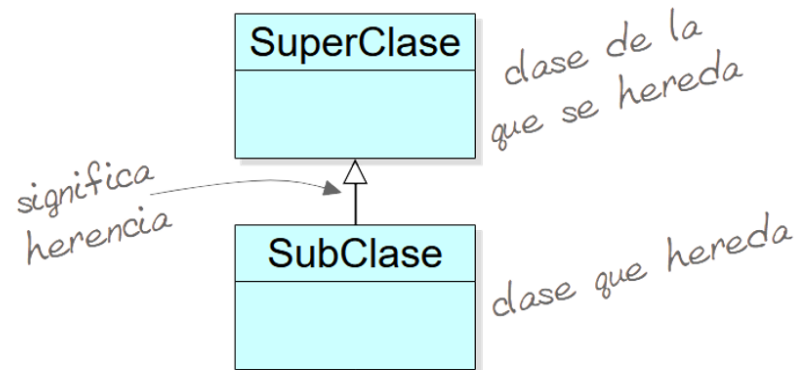
6

- ▶ Las clases (objetos) se pueden relacionar entre sí.
- ▶ Tipos de asociación (relación):
 - ▶ **Uso** : un objeto utiliza otro, por ejemplo en algún método del primer objeto, se instancia un objeto de otra clase (parámetro).
 - ▶ **Agregación:**
 - Se define en una clase al menos un atributo de un tipo de dato de otra clase.
 - Se conoce también como composición débil.
 - Los objetos atributo pueden existir independientemente.
 - Ejemplo: Carrito de la compra y productos.
 - ▶ **Composición**
 - ▶ **Herencia**

► Composición

- Una instancia de una clase está compuesta de instancias de la otra clase, y la segunda no puede existir si no es asociada a la primera.
- Se conoce como composición fuerte.
- Ejemplo: un hotel y sus habitaciones.

- ▶ La relación de **herencia** es una de las aportaciones más importantes de la POO.
- ▶ El paradigma POO facilita la reutilización del código y la herencia, además, permite crear componentes a partir de otros:
 - ▶ La herencia es el mecanismo que permite extender la estructura y/o funcionalidad de una clase a otra: se heredan los atributos y métodos visibles.



- ▶ Cuando se programa en java, se utiliza la herencia desde el primer momento, ya que todas las clases heredan de la clase **java.lang.Object**:
 - ▶ Métodos equals y hashCode
 - ▶ toString
 - ▶ ...
- ▶ Ejemplos de relaciones de herencia:

Superclase / padre	Subclase / hija
Estudiante	EstudianteUniversitario, EstudianteSecundaria
Empleado	Docente, Administrativo, JefeEstudios
Animal	Perro, Gato, Oca
Figura	Triangulo, Cuadrado, Circulo

- ▶ La asociación de herencia suele nombrarse como '**es-un**' (is-a). Por ejemplo, un docente es un empleado.
- ▶ Java no permite el uso de herencia múltiple, esto significa que **una subclase solo puede tener una superclase**. Y una superclase puede tener varias subclases.
- ▶ Las subclases heredan los atributos y métodos de la superclase, pero **no se heredan los constructores**
- ▶ Para indicar que una clase es subclase se utiliza la palabra reservada **extends**.

```
public class Perro extends Animal {  
    public Perro(String raza, double peso) {  
        super(raza, peso); // constructor de la superclase (Animal)  
    }  
}
```

► Modificadores de acceso

- La subclase hereda todos los miembros que no sean privados: un atributo privado en la superclase se puede acceder a través de un método público (get).
- Con el modificador **protected** en los miembros de la superclase se permite que estos sean **visibles desde las subclases** tanto si son vecinas como externas, en este último caso, se tendría que importar la superclase.

	Visible desde...			
	la propia clase	clases vecinas	Subclases	clases externas
private	✓			
sin modificador	✓	✓		
protected	✓	✓	✓	
public	✓	✓	✓	✓

- ▶ **Redefinición de miembros heredados:** Consiste en declarar en la subclase un miembro con igual nombre que uno heredado:
 - ▶ **Ocultación:** es la redefinición de un atributo en la subclase
 - ▶ **Overriding** : es la redefinición de un método en la subclase
 - Se suele señalar el método redefinido con @Override
 - La **sustitución** de un método de la superclase debe hacerse con el mismo **nombre** de método, misma lista de **parámetros** y mismo valor de **retorno**.

► **super y super()**

- **super:** para referenciar a la superclase, y con el punto '.' acceder a los miembros.
- **super()** : invoca al constructor de la superclase. Si se utiliza, debe ser la primera instrucción del constructor de la subclase.

```
public class Persona {  
    String nombre;  
    byte edad;  
    double estatura;  
  
    Persona (String nombre, byte edad, double estatura){  
        this.nombre = nombre;  
        this.edad = edad;  
        this.estatura = estatura;  
    }  
}
```

```
class Empleado extends Persona {  
    double salario;  
  
    Empleado (String nombre, byte edad, double estatura,  
              double salario) {  
        super(nombre, edad, estatura); //constructor de Persona  
        this.salario = salario; //atributo propio de Empleado  
    }  
}
```

- ▶ Cualquier clase puede heredar de otra, siempre que sea pública
- ▶ Para **impedir que una clase sea 'superclase'**, es decir, que no se pueda heredar de ella, hay que marcarla como **final**.

```
public final class Prueba { ...}
```

- ▶ Cuando se establece una relación de herencia entre clases, se puede instanciar objetos de la subclase utilizando como referencia la superclase: **Conversión hacia arriba**

```
Animal a = new Perro(); // solo se puede acceder a los métodos de Perro
                        // definidos en Animal
Perro p = new Perro();
```

- ▶ Esta característica es útil para crear por ejemplo arrays o métodos en los que se quiere combinar el uso de objetos de la superclase y subclases.

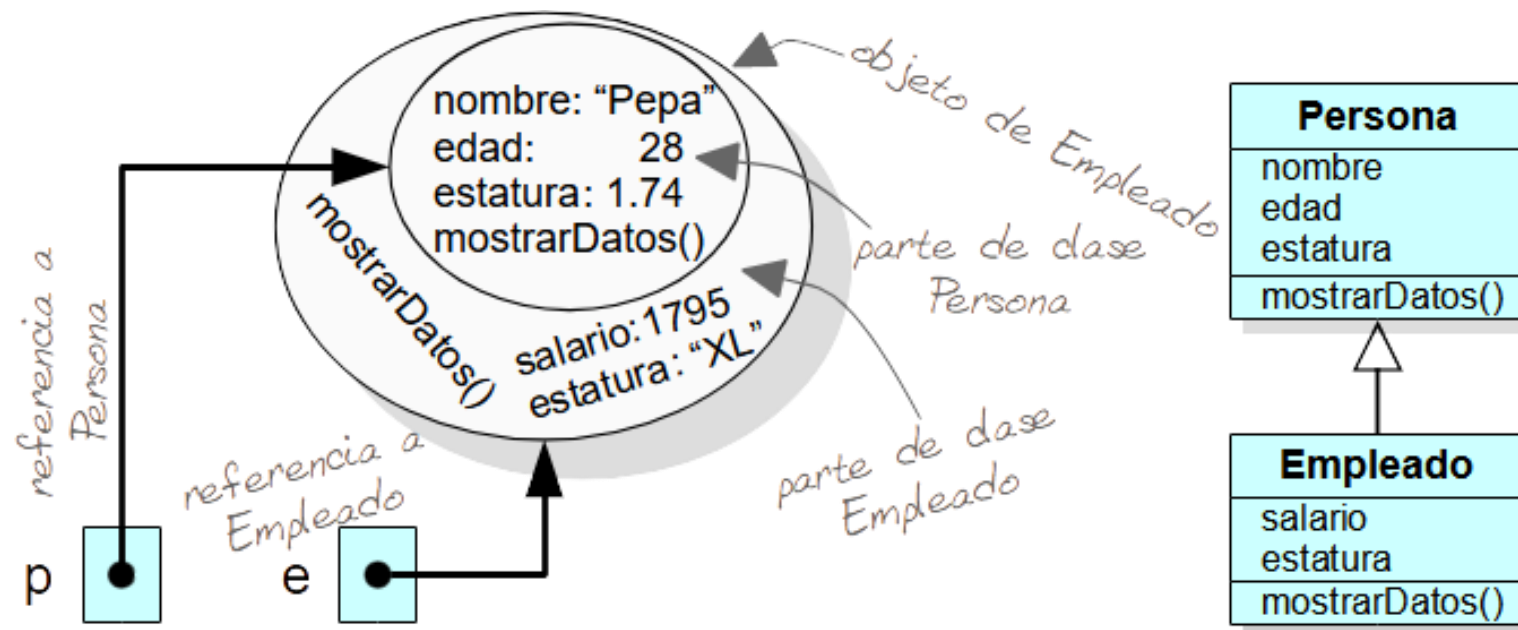
a)

```
Poligono [] figuras = new Poligono [3];
figuras[0]= new Poligono();
figuras[1]= new Triangulo();
figuras[2]= new Rectangulo();
```

b)

```
void calculaArea (Poligono p)
// se podría invocar calculaArea con un
Triangulo, Rectangulo, Poligono
```

- El **polimorfismo** se da al aplicar la 'conversión hacia arriba' combinada con la ocultación de métodos que se da cuando se sobrescriben en las subclases los métodos de la superclase



- ▶ Continuando con el escenario de la anterior figura, cuando en la subclase hay ocultación de atributos o sobrescritura de métodos de la superclase, ¿a qué miembros se accede, de Persona o de Empleado?

```
Empleado e = new Empleado();  
Persona p = e;
```

- ▶ Los **atributos** visibles son los definidos en la clase de la variable, entonces:

```
p.estatura // de Persona (double)  
e.estatura // de Empleado (String)
```

¡No se produce ocultación de atributos!

- ▶ Sin embargo, con los **métodos**, se ejecuta siempre el método más concreto, es decir, el método del objeto referenciado

```
p.mostrarDatos() // ¡¡de Empleado!!  
e.mostrarDatos() // de Empleado
```

- ▶ En el ejemplo, al llamar al método `p.mostrarDatos()`, gracias al **polimorfismo**, en tiempo de **ejecución**, si se detecta que:
 - La **referencia** (variable) es del tipo de la **superclase** (Persona) y
 - La **instancia** es del tipo de la **subclase** (Empleado) y
 - La subclase tiene una implementación del método que oculta (sobrescribe) la de la superclase
 - Se ejecuta el método más 'concreto', es decir, el de la subclase (Empleado)

- ▶ Al aplicar herencia puede interesar definir superclases para dar estructura base a las subclases pero que no se puedan instanciar.
- ▶ En la superclase que no se quiere instanciar se definen los miembros comunes a las subclases
- ▶ Una clase de este tipo es una clase abstracta y se identifica con la palabra reservada **'abstract'**
- ▶ Las clases abstractas tienen atributos y métodos que van a heredar las subclases.
- ▶ Los métodos de una clase abstracta pueden ser:
 - ▶ **Concretos:** se declaran e implementan normalmente
 - ▶ **Abstractos:** se declara únicamente el prototipo, añadiendo la palabra reservada **abstract** y se espera que la implementación se realice en las subclases.

- ▶ Los métodos abstractos se definen únicamente en clases abstractas
- ▶ Las subclases de una clase abstracta deben :
 - ▶ Implementar los métodos abstractos de la superclase (@Override)
 - ▶ O bien definirse también como abstractas y delegar la implementación del método abstracto a otras subclases.